

Spring 2026 Data C100/C200 Midterm 1 Reference Sheet

Pandas

Suppose `df` is a DataFrame; `s` is a Series. `import pandas as pd`

Function	Description
<code>df.shape</code>	Returns a tuple containing the number of rows and columns, in that order
<code>df.index</code>	Returns the index (row labels) of <code>df</code> as an Index object
<code>df[col]</code>	Returns the column labeled <code>col</code> from <code>df</code> as a Series
<code>df.index[i]</code>	Returns the row label at position <code>i</code> from <code>df</code> 's index
<code>df[[col1, col2]]</code>	Returns a DataFrame containing the columns labeled <code>col1</code> and <code>col2</code>
<code>s.idxmax()</code>	Returns the index label of the first occurrence of the maximum value in Series <code>s</code>
<code>s.astype(dtype)</code>	Returns a Series casted to the specified type <code>dtype</code>
<code>s.loc[rows] / df.loc[rows, cols]</code>	Returns a Series/DataFrame with rows (and columns) selected by their index values
<code>s.iloc[rows] / df.iloc[rows, cols]</code>	Returns a Series/DataFrame with rows (and columns) selected by their positions
<code>s.isnull() / df.isnull()</code>	Returns boolean Series/DataFrame identifying missing values
<code>s.fillna(value) / df.fillna(value)</code>	Returns a Series/DataFrame where missing values are replaced by <code>value</code>
<code>s.isin(values) / df.isin(values)</code>	Returns a Series/DataFrame of booleans indicating if each element is in <code>values</code> .
<code>df.drop(labels, axis)</code>	Returns a DataFrame without the rows or columns named <code>labels</code> along <code>axis</code> (either 0 or 1)
<code>df.rename(index=None, columns=None)</code>	Returns a DataFrame with renamed columns from a dictionary <code>index</code> and/or <code>columns</code>
<code>df.sort_values(by, ascending=True)</code>	Returns a DataFrame where rows are sorted by the values in columns <code>by</code>
<code>s.sort_values(ascending=True)</code>	Returns a sorted Series
<code>s.unique()</code>	Returns a NumPy array of the unique values of <code>s</code> in the order that they appear
<code>s.value_counts()</code>	Returns the number of times each unique value appears in a Series
<code>pd.merge(left, right, how='inner', left_on=col1, right_on=col2)</code>	Returns a DataFrame joining left and right on columns labeled <code>col1</code> and <code>col2</code> ; the join is of type inner
<code>left.merge(right, left_on=col1, right_on=col2)</code>	Returns a DataFrame joining <code>left</code> and <code>right</code> on columns labeled <code>col1</code> and <code>col2</code>
<code>df.pivot_table(values=None, index=None, columns=None, aggfunc='mean', fill_value=None)</code>	Returns a DataFrame pivot table where columns are unique values from <code>columns</code> (column name or list), and rows are unique values from <code>index</code> (column name or list); cells are collected <code>values</code> using <code>aggfunc</code> . If <code>values</code> is not provided, cells are collected for each remaining column with multi-level column indexing.
<code>df.set_index(col)</code>	Returns a DataFrame that uses the values in the column labeled <code>col</code> as the row index
<code>df.reset_index()</code>	Returns a DataFrame that has row index 0, 1, etc., and adds the current index as a column

Let `grouped = df.groupby(by)` where `by` can be a column label or a list of labels

Function	Description
<code>grouped.count()</code>	Return a DataFrame containing the size of each group, excluding missing values
<code>grouped.size()</code>	Return a Series containing size of each group, including missing values
<code>grouped.mean().min().max()</code>	Return a Series/DataFrame containing mean/min/max of each group for each column, excluding missing values
<code>grouped.head(n).tail(n)</code>	Return a Series/DataFrame containing first/last <code>n</code> entries of each group for each column, excluding missing values
<code>grouped.filter(f)</code> <code>grouped.agg(f)</code>	Filters or aggregates using the given function <code>f</code>
Function	Description
<code>s.str.len()</code>	Returns a Series containing length of each string

Function	Description
<code>s.str[a:b]</code>	Returns a Series where each element is a slice of the corresponding string indexed from a (inclusive, optional) to b (non-inclusive, optional)
<code>s.str.lower()/s.str.upper()</code>	Returns a Series of lowercase/uppercase versions of each string
<code>s.str.replace(pat, repl, regex=False)</code>	Returns a Series that replaces occurrences of substrings matching pat with string repl . When regex=False , pat is treated as a literal string; when regex=True , pat is treated as a RegEx pattern.
<code>s.str.contains(pat)</code>	Returns a boolean Series indicating if a substring matching the regex pat is contained in each string
<code>s.str.extract(pat)</code>	Returns a DataFrame of the first subsequence of each string that matches the regex pat . If pat contains one group, then only the substring matching the group is extracted
<code>s.str.split(pat=" ")</code>	Splits the strings in s at the delimiter pat (defaults to a whitespace). Returns a Series of lists, where each list contains strings of the characters before and after the split.

Visualization

Matplotlib: **x** and **y** are sequences of values. `import matplotlib.pyplot as plt`

Function	Description
<code>plt.plot(x, y)</code>	Creates a line plot of x against y
<code>plt.scatter(x, y)</code>	Creates a scatter plot of x against y
<code>plt.hist(x, bins=None)</code>	Creates a histogram of x ; bins can be an integer or a sequence
<code>plt.bar(x, height)</code>	Creates a bar plot of categories x and corresponding heights height

Seaborn: **x** and **y** are column names in a DataFrame **data**. `import seaborn as sns`

Function	Description
<code>sns.countplot(data=None, x=None)</code>	Create a barplot of value counts of variable x from data
<code>sns.histplot(data=None, x=None, stat='count', kde=False)</code> <code>sns.displot(data=None, x=None, kind='hist', rug=False)</code>	Creates a histogram of x from data , where bin statistics stat is one of 'count' , 'frequency' , 'probability' , 'percent' , and 'density' ; optionally overlay a kernel density estimator. displot is similar but can optionally overlay a rug plot and/or a KDE plot
<code>sns.rugplot(data=None, x=None)</code>	Adds a rug plot on the x-axis of variable x from data
<code>sns.boxplot(data=None, x=None, y=None)</code> <code>sns.violinplot(data=None, x=None, y=None)</code>	Create a boxplot of a numeric feature (e.g., y), optionally factoring by a category (e.g., x), from data . violinplot is similar but also draws a kernel density estimator of the numeric feature
<code>sns.scatterplot(data=None, x=None, y=None)</code>	Create a scatterplot of x versus y from data
<code>sns.lmplot(data=None, x=None, y=None, fit_reg=True)</code>	Create a scatterplot of x versus y from data , and by default overlay a least-squares regression line
<code>sns.jointplot(data=None, x=None, y=None, kind='scatter')</code>	Combine a bivariate scatterplot of x versus y from data , with univariate density plots of each variable overlaid on the axes; kind determines the visualization type for the distribution plot, can be scatter , kde or hist

Regular Expressions

Operator	Description	Operator	Description
<code>.</code>	Matches any character except <code>\n</code>	<code>*</code>	Matches preceding character/group zero or more times
<code>\</code>	Escapes metacharacters	<code>?</code>	Matches preceding character/group zero or one times
<code> </code>	Matches expression on either side of expression; has lowest priority of any operator	<code>+</code>	Matches preceding character/group one or more times
<code>\d, \w, \s</code>	Predefined character group of digits (0-9), alphanumerics (a-z, A-Z, 0-9, and underscore), or whitespace, respectively	<code>^, \$</code>	Matches the beginning and end of the line, respectively
<code>\D, \W, \S</code>	Inverse sets of <code>\d, \w, \s</code> , respectively	<code>()</code>	Capturing group used to create a sub-expression

Operator	Description	Operator	Description
{m}	Matches preceding character/group exactly m times	[]	Character class used to match any of the specified characters or range (e.g. [abcde] is equivalent to [a-e])
{m, n}	Matches preceding character/group at least m times and at most n times. If either m or n are omitted, set lower/upper bounds to 0 and ∞ , respectively	[^]	Invert character class; e.g. [^a-c] matches all characters except a, b, c

Modified lecture example for capture groups:

```
import re
lines = '169.237.46.168 - - [26/Jan/2014:10:47:58 -0800] "GET ... HTTP/1.1"'
re.findall(r'\[\d+\./(\w+)\.\d+:\d+:\d+:\d+ .+\]', lines) # returns ['Jan']
```

Function	Description
re.match(pattern, string)	Returns a match if zero or more characters at beginning of string matches pattern , else None
re.search(pattern, string)	Returns a match if zero or more characters anywhere in string matches pattern , else None
re.findall(pattern, string)	Returns a list of all non-overlapping matches of pattern in string (if none, returns empty list)
re.sub(pattern, repl, string)	Returns string after replacing all occurrences of pattern with repl